Министерство образования Республики Беларусь

Учреждение образования

«Брестский государственный технический университет»

Кафедра ИИТ

Лабораторная работа №2

По дисциплине «Современные методы защиты компьютерных систем»

Тема: «Защита веб-периметра и ограничение интенсивности сетевого трафика средствами межсетевых экранов прикладного уровня»

Выполнил:

Студент 4 курса

Группы ИИ-26

Рубцов Д.А.

Проверил:

Хацкевич А.С.

Брест 2026

Цель работы: практическое освоение методов развертывания межсетевых экранов прикладного уровня (WAF), написания пользовательских правил фильтрации L7-угроз, а также проектирование механизмов защиты от DDoS-атак и ограничения частоты запросов (Rate Limiting) на базе обратного прокси-сервера Nginx.

Вариант 14:

Вариант	WAF: Блокируемый User-Agent	ID правила WAF	Rate Limit: Эндпоинт (Target)	Rate Limit: rate	Rate Limit: burst
14	`Acunetix`	10014	`/api/password_reset`	`1r/s`	`1`

Задание 1: Базовая фильтрация угроз через WAF (OWASP Core Rule Set)

1) Curl запрос: curl -i http://127.0.0.1:5000/?id=1'%20OR%20'1'='1

```
HTTP/1.1 200 OK
Server: Werkzeug/3.1.9 Python/3.14.7
Date: Wed, 14 Oct 2026 10:52:43 GMT
Content-Type: text/html; charset=utf-8
Content-Length: 31
Connection: close

Backend OK. Endpoint reached: /[gragb0t@GragB0t ~]$
```

2) Запрос через Nginx: curl -i "http://127.0.0.1/?id=1'%20OR%20'1'='1"

```
HTTP/1.1 403 Forbidden
Server: nginx/1.30.5
Date: Wed, 14 Oct 2026 11:27:41 GMT
Content-Type: text/html
Content-Length: 153
Connection: keep-alive

<html>
<head><title>403 Forbidden</title></head>
<body>
<center><h1>403 Forbidden</h1></center>
<hr><center>nginx/1.30.5</center>
</body>
</html>
```

Задание 2: Написание пользовательских правил фильтрации (L7) (Acunetix /10014)

1) Использование пользовательского WAF правила:

```
SecRule REQUEST_HEADERS:User-Agent "@contains Acunetix" \
  "id:10014,\
  phase:1,\
  deny,\
  status:403,\
```

log,\
msg:'Access blocked for vulnerability scanner: Acunetix'"

```
HTTP/1.1 403 Forbidden
Server: nginx/1.30.5
Date: Wed, 14 Oct 2026 11:59:38 GMT
Content-Type: text/html
Content-Length: 153
Connection: keep-alive

<html>
<head><title>403 Forbidden</title></head>
<body>
<center><h1>403 Forbidden</h1></center>
<hr><center>nginx/1.30.5</center>
</body>
</html>
```

Задание 3: Защита от L7 DDoS-атак (Rate Limiting) (1r/s, burst limit 1, /api/password_reset)
1) Выполнение стресс теста ab -n 500 -c 10 http://127.0.0.1/api/password_reset

```
[gragb0t@GragB0t ~]$ ab -n 500 -c 10 http://127.0.0.1/api/password_reset
This is ApacheBench, Version 2.3 <$Revision: 1934973 $>
Copyright 1996 Adam Twiss, Zeus Technology Ltd, http://www.zeustech.net/
Licensed to The Apache Software Foundation, http://www.apache.org/

Benchmarking 127.0.0.1 (be patient)
Completed 100 requests
Completed 200 requests
Completed 300 requests
Completed 400 requests
Completed 500 requests
Finished 500 requests


Server Software:        nginx/1.30.5
Server Hostname:        127.0.0.1
Server Port:            80

Document Path:          /api/password_reset
Document Length:        49 bytes

Concurrency Level:      10
Time taken for tests:   2.367 seconds
Complete requests:      500
Failed requests:        0
Total transferred:      103000 bytes
HTML transferred:       24500 bytes
Requests per second:    211.23 [#/sec] (mean)
Time per request:       47.341 [ms] (mean)
Time per request:       4.734 [ms] (mean, across all concurrent requests)
Transfer rate:          42.49 [Kbytes/sec] received

Connection Times (ms)
              min  mean[+/-sd] median   max
Connect:        0    0   0.2      0       2
Processing:    16   46   9.6     46      77
Waiting:       13   45   9.4     44      75
Total:         16   47   9.6     46      77
```

2) Пофторный тест после установки ограничения интенсивности трафика (1r/s bust-1)
limit_req_zone $binary_remote_addr zone=mylimit:10m rate=1r/s;
limit_req zone=mylimit burst=1 nodelay;

```
[gragb0t@GragB0t ~]$ ab -n 500 -c 10 http://127.0.0.1/api/password_reset
This is ApacheBench, Version 2.3 <$Revision: 1934973 $>
Copyright 1996 Adam Twiss, Zeus Technology Ltd, http://www.zeustech.net/
Licensed to The Apache Software Foundation, http://www.apache.org/

Benchmarking 127.0.0.1 (be patient)
Completed 100 requests
Completed 200 requests
Completed 300 requests
Completed 400 requests
Completed 500 requests
Finished 500 requests


Server Software:        nginx/1.30.5
Server Hostname:        127.0.0.1
Server Port:            80

Document Path:          /api/password_reset
Document Length:        49 bytes

Concurrency Level:      10
Time taken for tests:   1.159 seconds
Complete requests:      500
Failed requests:        497
   (Connect: 0, Receive: 0, Length: 497, Exceptions: 0)
Non-2xx responses:      497
Total transferred:      184011 bytes
HTML transferred:       98056 bytes
Requests per second:    431.52 [#/sec] (mean)
Time per request:       23.174 [ms] (mean)
Time per request:       2.317 [ms] (mean, across all concurrent requests)
Transfer rate:          155.09 [Kbytes/sec] received

Connection Times (ms)
              min  mean[+/-sd] median   max
Connect:        0    0   0.2      0       2
Processing:     7   22   7.5     21      78
Waiting:        7   21   7.1     20      75
Total:          8   23   7.5     21      78
```

Задание 4: Проектирование профилей защиты (Без написания кода)

1) Составьте конфигурацию директив Nginx для защиты от атаки «Slowloris» (Slow HTTP Attack), указав оптимальные значения таймаутов (client_body_timeout, client_header_timeout) и обоснуйте свой выбор.

Client_body_timeout - Ограничевает время ожидания передачи тела запроса по стандарту его значение равно 60 сек что очень много и подвергает угрозе атакой Slowloris поэтому данное значение нужно снизить но при этом его нельзя снизить до минимума например 1 сек так как обычный пользователь с медленным подключением просто не успеет отправить тело запроса по этому его нужно установить на значении например 10 сек чтобы обычный легитивный пользовотель успел передать запрос но пресекать злоумышлиников.

Client_header_timeout - Время ожидания передачи заголовка запроса, аналогично 10 сек по тем же причинам.

2) Описание обработки трафика в эшелонированной защите

Уровень CDN (защита L3/L4): Запросы пользователей сначала попадают на распределенную сеть CDN (использующую технологию Anycast). CDN принимает на себя сетевые и транспортные атаки, фильтруя их на периметре с помощью глобальных центров очистки трафика, а также кентирует статический контент, разгружая инфраструктуру.

Уровень WAF (L7-защита): Очищенный от сетевого мусора трафик перенаправляется на шлюз WAF. Здесь происходит глубокий анализ HTTP-протокола: срабатывают сигнатуры OWASP CRS (защита от SQLi, XSS), проверяются пользовательские правила (блокировка вредоносных сканеров вроде Acunetix) и отрабатывают лимиты частоты запросов (Rate Limiting).

Уровень Приложения: До защитного контура доходят исключительно легитимные, проверенные и безопасные запросы, что гарантирует стабильную работу бизнес-логики и базы данных.

Вывод: на практическе освоил методы развертывания межсетевых экранов прикладного уровня (WAF), написание пользовательских правил фильтрации L7-угроз, а также проектировал механизмы защиты от DDoS-атак и ограничение частоты запросов (Rate Limiting) на базе обратного прокси-сервера Nginx.